mirafra
TECHNOLOGIES

# AXI4-Lite Master-Slave Communication System: Detailed Project Report

**Prepared by:** K Shreenidhi and Krishna Negi

# Table of Contents


**1. Abstract** **1**
**2. Introduction** **2**
**3. Project Objectives** **3**
**4. System Architecture** **3**
**5. AXI4-Lite Interface Signals** **5**
5.1 Global Signals 5
5.2 Write Address Channel 6
5.3 Write Data Channel 6
5.4 Write Response Channel 7
5.5 Read Address Channel 7
5.6 Read Data Channel 8
**6. Module-Level Implementation** **8**
6.1 AXI4-Lite Master 8
6.2 AXI4-Lite Slave 10
**7. Memory Organization and Byte Enable Support** **12**
7.1 Memory Organization 12
7.2 Byte Enable (WSTRB) Support 13
**8. Transaction Sequences and Error Handling** **13**
8.1 Parallel Operation 13
8.2 Error Handling Responses 13
8.3 Reset Behaviour 14
**9. Results and Discussion** **14**
9.1 Simulation Results 15
**10. Conclusion** **17**
**11. Appendix:** **18**
11.1 Verilog Source Code 18
11.2 Lint Reports 30
11.3 Timing Reports 32

# 1. Abstract

This document presents a comprehensive design and implementation of an AXI4-Lite Master–Slave communication system. The system demonstrates protocol-compliant read and write transactions using independent VALID/READY handshakes across all five AXI4-Lite channels. The master generates transactions while the slave implements a register-based memory with support for byte-enable writes and detailed error handling. Finite State Machines (FSMs) are used for both read and write channels to ensure correct sequencing. The design successfully models realistic SoC communication behavior including address validation, access restrictions, and response generation.

# 2. Introduction

AXI4-Lite is a simplified version of the AXI protocol used primarily for control and status register access in SoCs. It supports single data transfers and uses independent channels for communication.

This project implements:

- An AXI4-Lite Master, acting as the transaction generator.
- An AXI4-Lite Slave, acting as a memory-mapped peripheral.

The design emphasizes protocol correctness, modular design, and detailed handling of edge cases such as invalid addresses and restricted memory regions.

# 3. Project Objectives

- Implement an AXI4-Lite compliant master and slave.
- Support independent read and write channels.
- Implement FSM-based control.
- Support byte-enable writes using the WSTRB signal.
- Implement address validation.
- Generate correct AXI responses.
- Ensure robust and lossless communication.

# 4. System Architecture

The system consists of two primary modules operating synchronously using the global ACLK and active-low ARESETn signals:

1. **AXI4-Lite Master:** Generates read and write transactions and manages handshakes.
2. **AXI4-Lite Slave:** Receives transactions, performs memory operations, and responds to requests.

Data transfer occurs using the five independent AXI4-Lite channels:

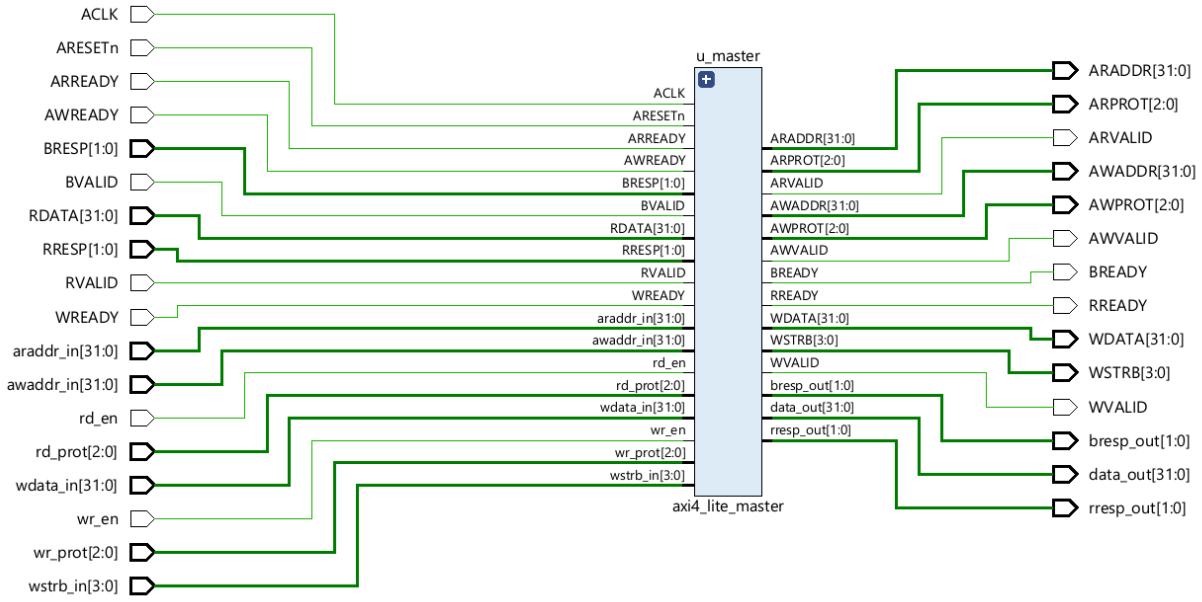
1. Write Address Channel
2. Write Data Channel
3. Write Response Channel
4. Read Address Channel

5. Read Data Channel

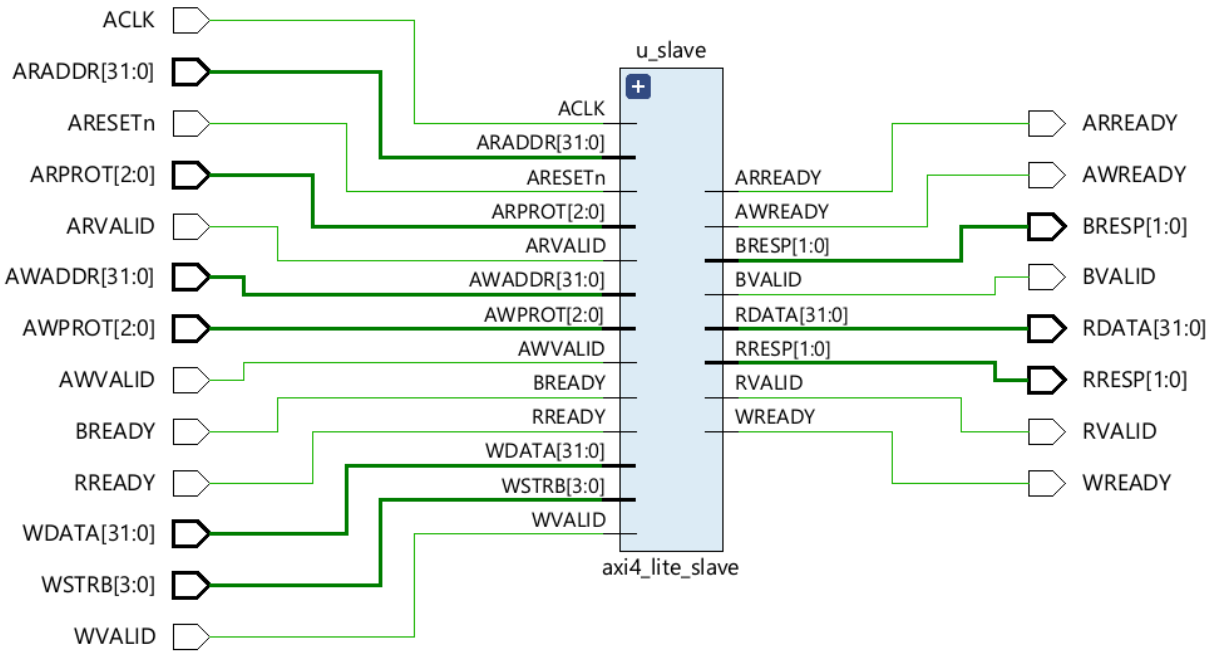
These channels operate using independent VALID-READY handshaking, which allows flexible transaction timing and back-pressure support. A transfer occurs only when the VALID and READY signals are high on the same clock edge.

## Block Design

### AXI4-Lite Master



**AXI4-Lite Slave**




# 5. AXI4-Lite Interface Signals

## 5.1 Global Signals

| Signal | Direction | Description |
|---|---|---|
| ACLK | Input | Global AXI clock |
| ARESETn | Input | Active-low reset |

## 5.2 Write Address Channel

| Signal | Direction | Description |
|---|---|---|
| AWADDR | Master -> Slave | Write address |
| AWPROT | Master -> Slave | Protection attributes |
| AWVALID | Master -> Slave | Address valid |
| AWREADY | Slave -> Master | Address accepted |

## 5.3 Write Data Channel

| Signal | Direction | Description |
|---|---|---|
| WDATA | Master -> Slave | Write data |
| WSTRB | Master -> Slave | Byte enables |
| WVALID | Master -> Slave | Write data valid |
| WREADY | Slave -> Master | Data accepted |

## 5.4 Write Response Channel

| Signal | Direction | Description |
|---|---|---|
| BRESP | Slave -> Master | Write response |
| BVALID | Slave -> Master | Response valid |
| BREADY | Master -> Slave | Response accepted |

## 5.5 Read Address Channel

| Signal | Direction | Description |
|---|---|---|
| ARADDR | Master -> Slave | Read address |
| ARPROT | Master -> Slave | Protection attributes |
| ARVALID | Master -> Slave | Address valid |
| ARREADY | Slave -> Master | Address accepted |

## 5.6 Read Data Channel

| Signal | Direction | Description |
|---|---|---|
| RDATA | Slave -> Master | Read data |
| RRESP | Slave -> Master | Read response |
| RVALID | Slave -> Master | Data valid |
| RREADY | Master -> Slave | Data accepted |

# 6. Module-Level Implementation

## 6.1 AXI4-Lite Master

The Master module is responsible for generating AXI transactions and managing handshake signals.

**Write FSM:** Controlled by a five-state FSM that handles the independent arrival of AWREADY and WREADY.

- **W_IDLE:** Wait for write request.
- **W_BOTH:** Send address and data.
- **W_ADDR:** Waiting for address handshake.

- **W_DATA:** Waiting for data handshake.
- **W_RESP:** Waiting for write response.

**Master Write FSM (5 States)**

This FSM generates write requests and handles the fact that the Slave might accept the address and the data at different times.

- **W_IDLE**: The resting state. The FSM waits here until the wr_en (write enable) signal is triggered by the user. Once triggered, it latches the target address and data, asserts both AWVALID and WVALID, and jumps to W_BOTH.
- **W_BOTH**: The Master is currently waiting for the Slave to accept *both* the address and the data.
    - If the Slave asserts AWREADY and WREADY at the same time, the FSM skips straight to W_RESP.
    - If the Slave accepts the address first (AWREADY is high, WREADY is low), it transitions to W_DATA.
    - If the Slave accepts the data first (WREADY is high, AWREADY is low), it transitions to W_ADDR.
- **W_DATA**: The address was accepted, but the Master is still holding WVALID high waiting for the Slave to assert WREADY. Once asserted, it moves to W_RESP.
- **W_ADDR**: The data was accepted, but the Master is still holding AWVALID high waiting for the Slave to assert AWREADY. Once asserted, it moves to W_RESP.
- **W_RESP**: The Master asserts BREADY, telling the Slave it is ready to receive the transaction status. Once the Slave asserts BVALID, the Master captures the BRESP (response code) and returns to W_IDLE.

**Read FSM:** Controlled by a three-state FSM.

- **R_IDLE:** Wait for read request.
- **R_ADDR:** Wait for slave ready.
- **R_DATA:** Wait for read data. Captures RDATA and RRESP when available.

**Master Read FSM (3 States)**

The read process is much simpler since the Master only needs to send an address and wait for the data to come back.

- **R_IDLE**: The FSM waits for the rd_en (read enable) signal. When triggered, it latches the read address, asserts ARVALID, and moves to R_ADDR.
- **R_ADDR**: The Master asserts RREADY (preparing to receive data) and waits for the Slave to assert ARREADY. Once the address is accepted, ARVALID is dropped, and the FSM moves to R_DATA.
- **R_DATA**: The Master waits for the Slave to assert RVALID. Once it does, the Master captures the read data (RDATA) and the response code (RRESP), then drops RREADY and returns to R_IDLE.

## 6.2 AXI4-Lite Slave

The Slave module handles incoming transactions and performs memory operations.

**Write FSM:** Supports decoupled address and data channels. It latches address and data independently before performing the write and generating a response.

- **W_IDLE:** Ready to accept address and data.
- **W_BOTH:** Waiting for both address and data.
- **W_ADDR:** Address received first.
- **W_DATA:** Data received first.
- **W_RESP:** Generate write response.

**Slave Write FSM (5 States)**

The Slave Write FSM mirrors the Master Write FSM. Its primary job is to safely latch incoming addresses and data, even if they arrive on different clock cycles, before executing the actual memory write.

- **W_IDLE**: The Slave asserts AWREADY and WREADY to indicate it is ready to accept a new transaction. If it sees either AWVALID or WVALID go high, it moves to W_BOTH.
- **W_BOTH**: The Slave uses two internal flags (aw_done and w_done) to track what it has received.
    - If it successfully handshakes the address, it latches AWADDR into wr_addr_lat and sets aw_done = 1.
    - If it successfully handshakes the data, it latches WDATA and WSTRB into wr_data_lat and wr_strb_lat, setting w_done = 1.
    - It drops its READY signals as soon as the respective data is latched to prevent overwriting. Once both flags are set, it moves to W_RESP.
- **W_ADDR**: It has the data but is still waiting for the address to become valid. Once AWVALID is high, it moves to W_RESP.
- **W_DATA**: It has the address but is still waiting for the data to become valid. Once WVALID is high, it moves to W_RESP.
- **W_RESP**: This is where the actual logic execution happens.
    - The Slave checks if the address is valid (DECERR if out of bounds) or if the user is trying to write to a read-only memory space (SLVERR).
    - If the address is completely valid, it writes the latched data into the memory array (reg_file), applying the byte-enable logic (WSTRB).
    - It outputs the appropriate BRESP code, asserts BVALID, and waits for the Master's BREADY before returning to W_IDLE and resetting its done flags.

**Read FSM:** A two-state FSM that validates the address and returns data or an error response.

- **R_IDLE:** Wait for read request.
- **R_DATA:** Send read data.

**Slave Read FSM (2 States)**

This is a very streamlined FSM dedicated to fetching data and reporting read errors.

- **R_IDLE**: The Slave asserts ARREADY. When the Master asserts ARVALID, the Slave latches the address, drops ARREADY, and immediately moves to R_DATA.
- **R_DATA**: The Slave evaluates the latched address:
    - If the address is out of bounds, it sets RRESP to DECERR and outputs 0 for

data.
- If the address points to a write-only register or is unaligned, it sets RRESP to SLVERR and outputs 0 for data.
- If the address is valid, it fetches the data from the reg_file and sets RRESP to OKAY.
- It then asserts RVALID. Once the Master accepts the data (RREADY is high), the Slave drops RVALID and goes back to R_IDLE.

# 7. Memory Organization and Byte Enable Support

## 7.1 Memory Organization

The slave implements a register file with a configurable depth (default MEM_DEPTH = 16). Each memory location stores 32-bit data. AXI uses byte addressing, while the internal memory uses word addressing. All accesses must be 32-bit aligned.

The address regions are mapped as follows:

| Byte Address Range | Word Index | Access Type | Description |
|---|---|---|---|
| 0x00 - 0x24 | 0 - 9 | Read/Write | Normal registers |
| 0x28 - 0x30 | 10 - 12 | Read-Only | Status registers |
| 0x34 - 0x38 | 13 - 14 | Write-Only | Command registers |
| 0x3C | 15 | Read/Write | Reserved/normal |

| | | | |
|---|---|---|---|
| Others | N/A | Invalid | Generates DECERR |

## 7.2 Byte Enable (WSTRB) Support

The design supports partial word updates using the WSTRB signal. This allows selective byte updates without overwriting the entire register.

- WSTRB[0] affects Bits [7:0]
- WSTRB[1] affects Bits [15:8]
- WSTRB[2] affects Bits [23:16]
- WSTRB[3] affects Bits [31:24]

# 8. Transaction Sequences and Error Handling

## 8.1 Parallel Operation

The read and write channels operate independently, allowing parallel read and write operations.

## 8.2 Error Handling Responses

The slave module generates AXI responses to indicate transaction success or failure.

- **OKAY (00):** Successful transaction.
- **SLVERR (10):** Generated for unaligned addresses, writes to a read-only region, or

reads to a write-only region.

- **DECERR (11):** Generated when an address is out of range.

## 8.3 Reset Behaviour

When ARESETn is asserted low, all interface VALID signals are cleared, READY signals return to default states, FSMs return to IDLE, and internal memory elements are reset to zero.

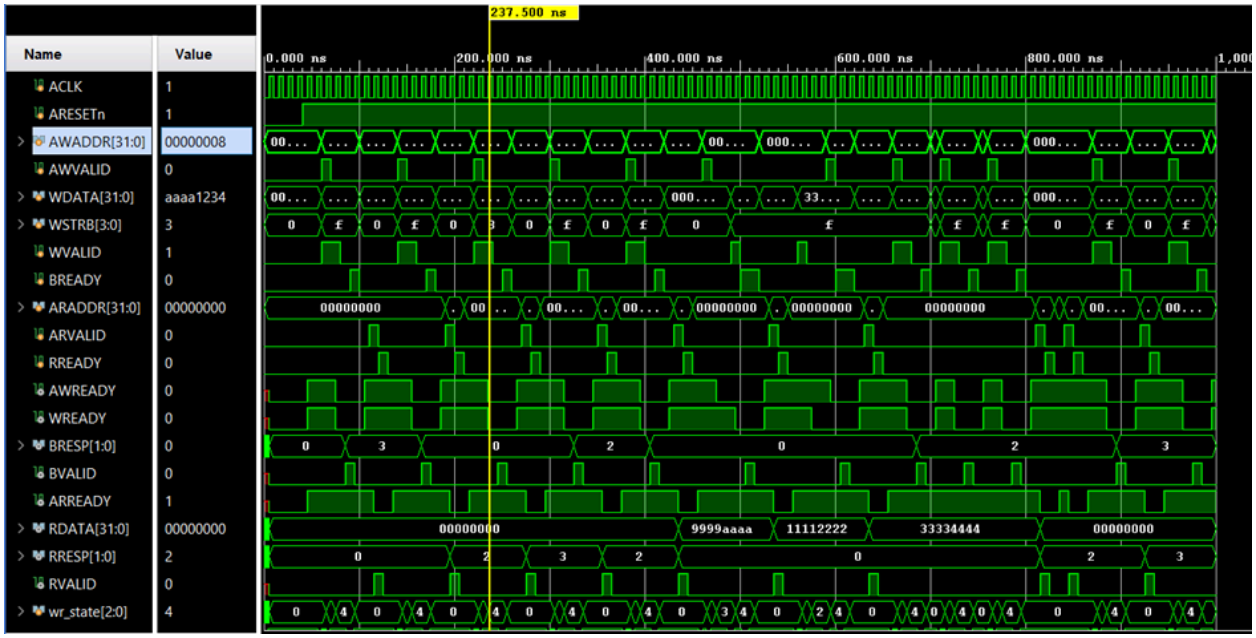
# 9. Results and Discussion

The system was simulated and demonstrated full AXI4-Lite compliance.

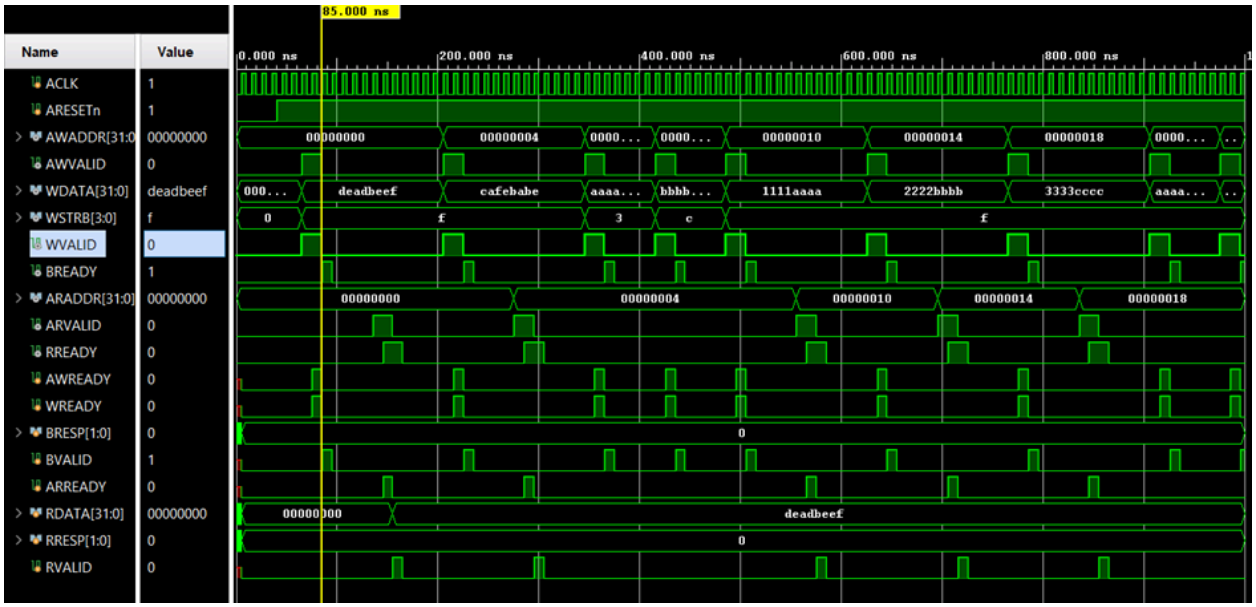
1. **Reset Behavior:** All FSMs initialized to IDLE and outputs reset to defaults.
2. **Write Operation:** Data was successfully written when both address and data were valid, and partial writes were properly handled via WSTRB.
3. **Read Operation:** Correct data was returned for valid addresses, and appropriate errors were flagged for invalid accesses.
4. **System Behavior:** Proper handshaking was maintained without data corruption, confirming reliable error reporting and correct FSM transitions.

# 9.1 Simulation Results

### Slave IP



## Master IP


85.000 ns
Name
Value
0.000 ns
200.000 ns
400.000 ns
600.000 ns
800.000 ns
ACLK
1
ARESETn
1
AWADDR[31:0]
00000000
AWVALID
0
WDATA[31:0]
deadbeef
WSTRB[3:0]
f
WVALID
0
BREADY
1
ARADDR[31:0]
00000000
ARVALID
0
RREADY
0
AWREADY
0
WREADY
0
BRESP[1:0]
0
BVALID
1
ARREADY
0
RDATA[31:0]
00000000
RRESP[1:0]
0
RVALID
0
00000000
00000004
0000...
00000010
00000014
00000018
deadbeef
cafebabe
aaaa...
bbbb...
1111aaaa
2222bbbb
3333cccc
f
3
c

# 10. Conclusion

This project successfully implements a complete AXI4-Lite Master–Slave system with detailed protocol adherence. The design demonstrates correct FSM-based control, memory-mapped communication, and robust error handling. It serves as a strong foundation for real-world SoC peripheral design.

---

# 11. Appendix:

## 11.1 Verilog Source Code

Verilog

```
module axi4_lite_master #(
    parameter DATA_WIDTH = 32,
    parameter ADDR_WIDTH = 32,
    parameter [2:0] DEFAULT_PROT = 3'b000
)(
    input  wire                     ACLK,
    input  wire                     ARESETn,

    output reg  [ADDR_WIDTH-1:0]    AWADDR,
    output reg  [2:0]               AWPROT,
    output reg                      AWVALID,
    input  wire                     AWREADY,

    output reg  [DATA_WIDTH-1:0]    WDATA,
    output reg  [(DATA_WIDTH/8)-1:0] WSTRB,
    output reg                      WVALID,
    input  wire                     WREADY,

    input  wire [1:0]               BRESP,
    input  wire                     BVALID,
    output reg                      BREADY,

    output reg  [ADDR_WIDTH-1:0]    ARADDR,
    output reg  [2:0]               ARPROT,
    output reg                      ARVALID,
    input  wire                     ARREADY,

    input  wire [DATA_WIDTH-1:0]    RDATA,
    input  wire [1:0]               RRESP,
    input  wire                     RVALID,
    output reg                      RREADY,
```

```
    input  wire                  wr_en,
    input  wire [ADDR_WIDTH-1:0]    awaddr_in,
    input  wire [DATA_WIDTH-1:0]    wdata_in,
    input  wire [(DATA_WIDTH/8)-1:0] wstrb_in,
    input  wire [2:0]            wr_prot,
    output reg  [1:0]             bresp_out,

    input  wire                  rd_en,
    input  wire [ADDR_WIDTH-1:0]    araddr_in,
    input  wire [2:0]            rd_prot,
    output reg  [DATA_WIDTH-1:0]    data_out,
    output reg  [1:0]             rresp_out
);

localparam [1:0]
    RESP_OKAY   = 2'b00,
    RESP_EXOKAY = 2'b01,
    RESP_SLVERR = 2'b10,
    RESP_DECERR = 2'b11;

localparam [2:0]
    W_IDLE = 3'd0,
    W_BOTH = 3'd1,
    W_ADDR = 3'd2,
    W_DATA = 3'd3,
    W_RESP = 3'd4;

reg [2:0] wr_state, n_wr_state;

always @(posedge ACLK or negedge ARESETn) begin
    if (!ARESETn)
        wr_state <= W_IDLE;
    else
        wr_state <= n_wr_state;
end

always @(*) begin
    n_wr_state = wr_state;
    case (wr_state)
        W_IDLE: if (wr_en) n_wr_state = W_BOTH;

        W_BOTH: begin
            if (AWREADY && WREADY)
```

```
            n_wr_state = W_RESP;
        else if (AWREADY && !WREADY)
            n_wr_state = W_DATA;
        else if (!AWREADY && WREADY)
            n_wr_state = W_ADDR;
    end

    W_ADDR: if (AWREADY) n_wr_state = W_RESP;
    W_DATA: if (WREADY)  n_wr_state = W_RESP;

    W_RESP: if (BVALID && BREADY) n_wr_state = W_IDLE;

    default: n_wr_state = W_IDLE;
  endcase
end

always @(posedge ACLK or negedge ARESETn) begin
  if (!ARESETn) begin
    AWADDR   <= 0;
    AWPROT   <= DEFAULT_PROT;
    AWVALID  <= 0;
    WDATA    <= 0;
    WSTRB    <= 0;
    WVALID   <= 0;
    BREADY   <= 0;
    bresp_out <= RESP_OKAY;
  end else begin
    case (wr_state)

        W_IDLE: begin
            AWVALID <= 0;
            WVALID  <= 0;
            BREADY  <= 0;

            if (wr_en) begin
                AWADDR  <= awaddr_in;
                AWPROT  <= wr_prot;
                AWVALID <= 1'b1;

                WDATA   <= wdata_in;
                WSTRB   <= wstrb_in;
                WVALID  <= 1'b1;
            end
```

```
        end

        W_BOTH: begin
          if (AWREADY && WREADY) begin
            AWVALID <= 0;
            WVALID  <= 0;
            BREADY  <= 1'b1;
          end else if (AWREADY && !WREADY) begin
            AWVALID <= 0;
          end else if (!AWREADY && WREADY) begin
            WVALID  <= 0;
          end
        end

        W_ADDR: begin
          if (AWREADY) begin
            AWVALID <= 0;
            BREADY  <= 1'b1;
          end
        end

        W_DATA: begin
          if (WREADY) begin
            WVALID <= 0;
            BREADY <= 1'b1;
          end
        end

        W_RESP: begin
          if (BVALID && BREADY) begin
            bresp_out <= BRESP;
            BREADY    <= 0;
          end
        end

      endcase
    end
end

localparam [1:0]
  R_IDLE = 2'd0,
  R_ADDR = 2'd1,
  R_DATA = 2'd2;
```

```
reg [1:0] rd_state, n_rd_state;

always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn)
      rd_state <= R_IDLE;
   else
      rd_state <= n_rd_state;
end

always @(*) begin
   n_rd_state = rd_state;
   case (rd_state)
      R_IDLE: if (rd_en) n_rd_state = R_ADDR;
      R_ADDR: if (ARREADY) n_rd_state = R_DATA;
      R_DATA: if (RVALID && RREADY) n_rd_state = R_IDLE;
      default: n_rd_state = R_IDLE;
   endcase
end

always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn) begin
      ARADDR   <= 0;
      ARPROT   <= DEFAULT_PROT;
      ARVALID  <= 0;
      RREADY   <= 0;
      data_out <= 0;
      rresp_out <= RESP_OKAY;
   end else begin
      case (rd_state)

         R_IDLE: begin
            ARVALID <= 0;
            RREADY  <= 0;

            if (rd_en) begin
               ARADDR  <= araddr_in;
               ARPROT  <= rd_prot;
               ARVALID <= 1'b1;
            end
         end

         R_ADDR: begin
```

```
            RREADY <= 1'b1;
            if (ARREADY)
               ARVALID <= 0;
         end

         R_DATA: begin
            if (RVALID && RREADY) begin
               data_out  <= RDATA;
               rresp_out <= RRESP;
               RREADY    <= 0;
            end
         end

      endcase
   end
end

endmodule

module axi4_lite_slave #(
   parameter DATA_WIDTH = 32,
   parameter ADDR_WIDTH = 32,
   parameter MEM_DEPTH  = 16,
   parameter [2:0] DEFAULT_PROT = 3'b000
)(
   input  wire                  ACLK,
   input  wire                  ARESETn,

   input  wire [ADDR_WIDTH-1:0]     AWADDR,
   input  wire [2:0]                AWPROT,
   input  wire                  AWVALID,
   output reg                    AWREADY,

   input  wire [DATA_WIDTH-1:0]     WDATA,
   input  wire [(DATA_WIDTH/8)-1:0]  WSTRB,
   input  wire                  WVALID,
   output reg                    WREADY,

   output reg  [1:0]                BRESP,
   output reg                    BVALID,
   input  wire                  BREADY,

   input  wire [ADDR_WIDTH-1:0]     ARADDR,
```

```
    input  wire [2:0]             ARPROT,
    input  wire                   ARVALID,
    output reg                    ARREADY,

    output reg  [DATA_WIDTH-1:0]  RDATA,
    output reg  [1:0]             RRESP,
    output reg                    RVALID,
    input  wire                   RREADY
);


localparam [1:0]
    RESP_OKAY   = 2'b00,
    RESP_EXOKAY = 2'b01,
    RESP_SLVERR = 2'b10,
    RESP_DECERR = 2'b11;

wire [2:0] slave_prot = DEFAULT_PROT;

integer i;
reg [DATA_WIDTH-1:0] reg_file [0:MEM_DEPTH-1];


localparam [2:0]
    W_IDLE = 3'd0,
    W_BOTH = 3'd1,
    W_ADDR = 3'd2,
    W_DATA = 3'd3,
    W_RESP = 3'd4;

reg [2:0] wr_state, n_wr_state;
reg aw_done, w_done;

reg [ADDR_WIDTH-1:0]  wr_addr_lat;
reg [DATA_WIDTH-1:0]  wr_data_lat;
reg [(DATA_WIDTH/8)-1:0] wr_strb_lat;
reg [2:0]             wr_prot_lat;


wire [$clog2(MEM_DEPTH)-1:0] wr_word_addr;
assign wr_word_addr = AWADDR[2 + $clog2(MEM_DEPTH)-1 : 2];

wire wr_addr_unaligned;
```

```
assign wr_addr_unaligned = |AWADDR[1:0];

wire wr_addr_ok;
assign wr_addr_ok = (wr_word_addr < MEM_DEPTH) && (AWADDR < (MEM_DEPTH * 4));

wire wr_addr_ro;
assign wr_addr_ro = (wr_word_addr >= 10) && (wr_word_addr <= 12);


always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn) begin
      aw_done     <= 1'b0;
      w_done      <= 1'b0;
      wr_addr_lat <= {ADDR_WIDTH{1'b0}};
      wr_data_lat <= {DATA_WIDTH{1'b0}};
      wr_strb_lat <= {(DATA_WIDTH/8){1'b0}};

      wr_prot_lat <= 3'b000;
   end else begin
      if (wr_state == W_RESP && BVALID && BREADY) begin
         aw_done <= 1'b0;
         w_done  <= 1'b0;
      end

     else if (AWVALID && AWREADY) begin
         aw_done     <= 1'b1;
         wr_addr_lat <= AWADDR;
         wr_prot_lat <= AWPROT;

      end
     else if (WVALID && WREADY) begin
         w_done      <= 1'b1;
         wr_data_lat <= WDATA;
         wr_strb_lat <= WSTRB;
      end
   end
end


always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn) wr_state <= W_IDLE;
   else          wr_state <= n_wr_state;
end
```

```
always @(*) begin
   n_wr_state = wr_state;
   case (wr_state)
      W_IDLE: if (AWVALID || WVALID) n_wr_state = W_BOTH;
      W_BOTH: begin
         if ((aw_done || (AWVALID && AWREADY)) &&
            (w_done  || (WVALID  && WREADY ))) n_wr_state = W_RESP;
         else if (aw_done || (AWVALID && AWREADY)) n_wr_state = W_DATA;
         else if (w_done  || (WVALID  && WREADY))  n_wr_state = W_ADDR;
      end
      W_ADDR: if (AWVALID && AWREADY) n_wr_state = W_RESP;
      W_DATA: if (WVALID  && WREADY)  n_wr_state = W_RESP;
      W_RESP: if (BVALID  && BREADY)  n_wr_state = W_IDLE;
      default:                        n_wr_state = W_IDLE;
   endcase
end


always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn) begin
      AWREADY <= 1'b0;
      WREADY  <= 1'b0;
      BVALID  <= 1'b0;
      BRESP   <= RESP_OKAY;
      for (i = 0; i < MEM_DEPTH; i = i + 1)
         reg_file[i] <= {DATA_WIDTH{1'b0}};
   end else begin
      case (wr_state)
         W_IDLE: begin
            AWREADY <= 1'b1;
            WREADY  <= 1'b1;
            BVALID  <= 1'b0;
         end
         W_BOTH: begin
            if (aw_done || (AWVALID && AWREADY))
               AWREADY <= 1'b0;
            if (w_done  || (WVALID  && WREADY ))  WREADY  <= 1'b0;
         end
         W_ADDR: begin
            AWREADY <= 1'b1;
            if (AWVALID && AWREADY) AWREADY <= 1'b0;
```

```
        end
        W_DATA: begin
          WREADY <= 1'b1;
          if (WVALID && WREADY) WREADY <= 1'b0;
        end
        W_RESP: begin
          if (!BVALID) begin
            if (!wr_addr_ok) begin
              BRESP  <= RESP_DECERR;
              reg_file[AWADDR[$clog2(MEM_DEPTH)+1:2]] <= 0;
            end else if (wr_addr_ro || wr_addr_unaligned) begin
              BRESP  <= RESP_SLVERR;
              reg_file[AWADDR[$clog2(MEM_DEPTH)+1:2]] <= 0;
            end else begin
              if (wr_strb_lat[0]) reg_file[wr_addr_lat[$clog2(MEM_DEPTH)+1:2]][ 7: 0] <=
wr_data_lat[ 7: 0];
              if (wr_strb_lat[1]) reg_file[wr_addr_lat[$clog2(MEM_DEPTH)+1:2]][15: 8] <=
wr_data_lat[15: 8];
              if (wr_strb_lat[2]) reg_file[wr_addr_lat[$clog2(MEM_DEPTH)+1:2]][23:16] <=
wr_data_lat[23:16];
              if (wr_strb_lat[3]) reg_file[wr_addr_lat[$clog2(MEM_DEPTH)+1:2]][31:24] <=
wr_data_lat[31:24];
              BRESP  <= RESP_OKAY;
            end
            BVALID <= 1'b1;
          end
          else if (BVALID && BREADY) BVALID <= 1'b0;
          else begin
            reg_file[AWADDR[$clog2(MEM_DEPTH)+1:2]] <=
reg_file[AWADDR[$clog2(MEM_DEPTH)+1:2]];
          end
        end
      endcase
    end
end


localparam
    R_IDLE = 1'd0,
    R_DATA = 1'd1;

reg  rd_state, n_rd_state;
reg [ADDR_WIDTH-1:0] rd_addr_lat;
```

```
reg [2:0]          rd_prot_lat;


wire [$clog2(MEM_DEPTH)-1:0] rd_word_idx = rd_addr_lat[$clog2(MEM_DEPTH)+1:2];

wire [$clog2(MEM_DEPTH)-1:0] rd_word_addr;
assign rd_word_addr = ARADDR[2 + $clog2(MEM_DEPTH)-1 : 2];

wire rd_addr_unaligned;
assign rd_addr_unaligned = |ARADDR[1:0];

wire rd_addr_ok;
assign rd_addr_ok = (rd_word_addr < MEM_DEPTH) && (ARADDR < (MEM_DEPTH * 4));

wire rd_addr_wo;
assign rd_addr_wo = (rd_word_addr >= 13) && (rd_word_addr <= 14);

always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn) rd_state <= R_IDLE;
   else          rd_state <= n_rd_state;
end


always @(*) begin
   n_rd_state = rd_state;

   case (rd_state)
      R_IDLE:  if (ARVALID && ARREADY)   n_rd_state = R_DATA;
      R_DATA:  if (RVALID && RREADY)     n_rd_state = R_IDLE;
      default:                           n_rd_state = R_IDLE;
   endcase
end


always @(posedge ACLK or negedge ARESETn) begin
   if (!ARESETn) begin
      ARREADY     <= 1'b0;
      RVALID      <= 1'b0;
      RDATA       <= {DATA_WIDTH{1'b0}};
      RRESP       <= RESP_OKAY;
      rd_addr_lat <= {ADDR_WIDTH{1'b0}};
      rd_prot_lat <= 3'b000;
   end else begin
```

```
    case (rd_state)

        R_IDLE: begin
            ARREADY <= 1'b1;
            RVALID  <= 1'b0;

            if (ARVALID && ARREADY) begin
                rd_addr_lat <= ARADDR;
                rd_prot_lat <= ARPROT;


                if (!rd_addr_ok)begin
                    RRESP <= RESP_DECERR;
                    RDATA <= 0;
                end
                else if (rd_addr_wo || rd_addr_unaligned) begin
                    RRESP <= RESP_SLVERR;
                    RDATA <= 0;
                end
                else begin
                    RRESP <= RESP_OKAY;
                    RDATA <= reg_file[ARADDR[$clog2(MEM_DEPTH)+1:2]];

                end
                RVALID <= 1'b1;
                ARREADY <= 1'b0;
            end
        end

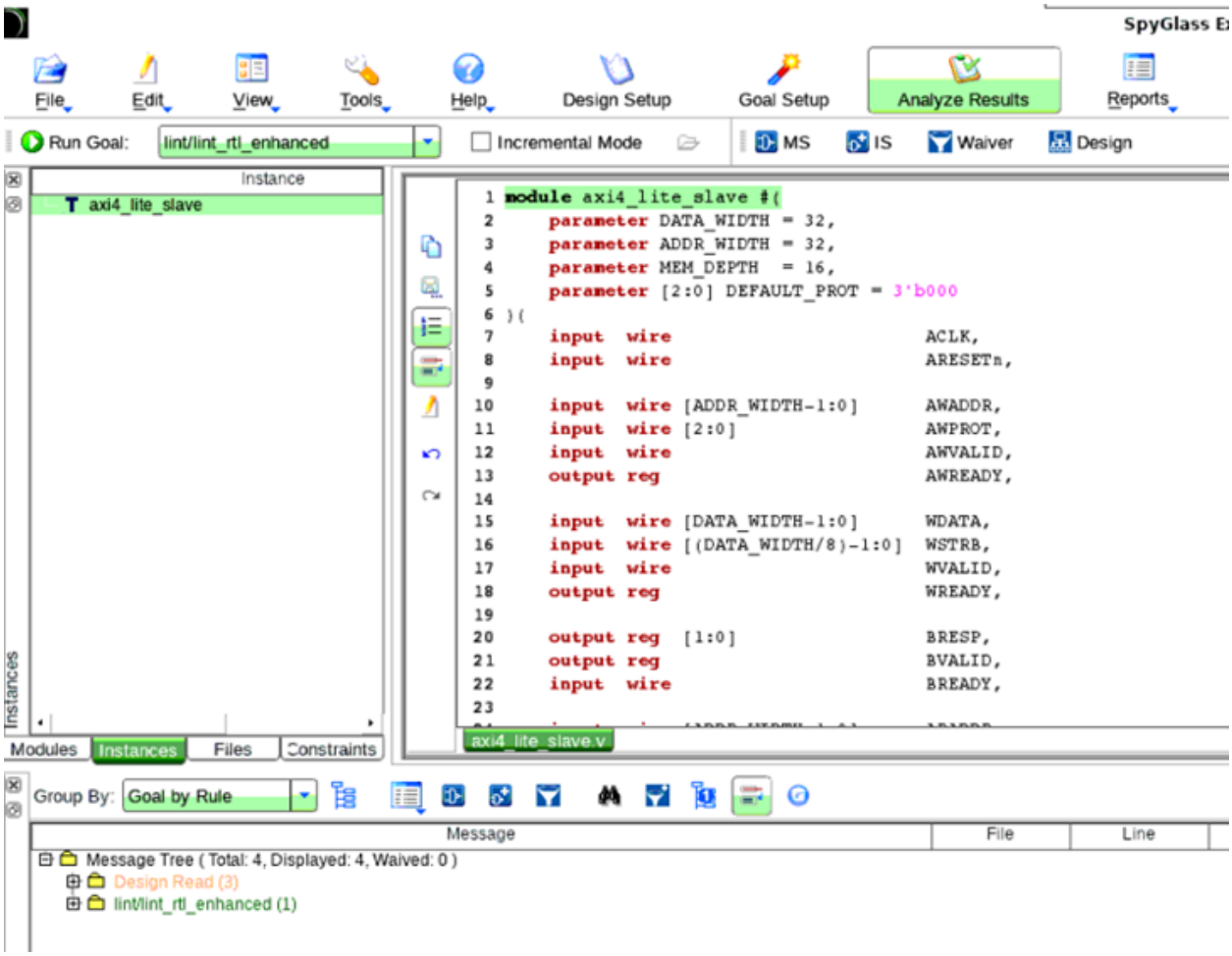

        R_DATA: begin
            if (RVALID && RREADY)
                RVALID <= 1'b0;
        end

    endcase
  end
end

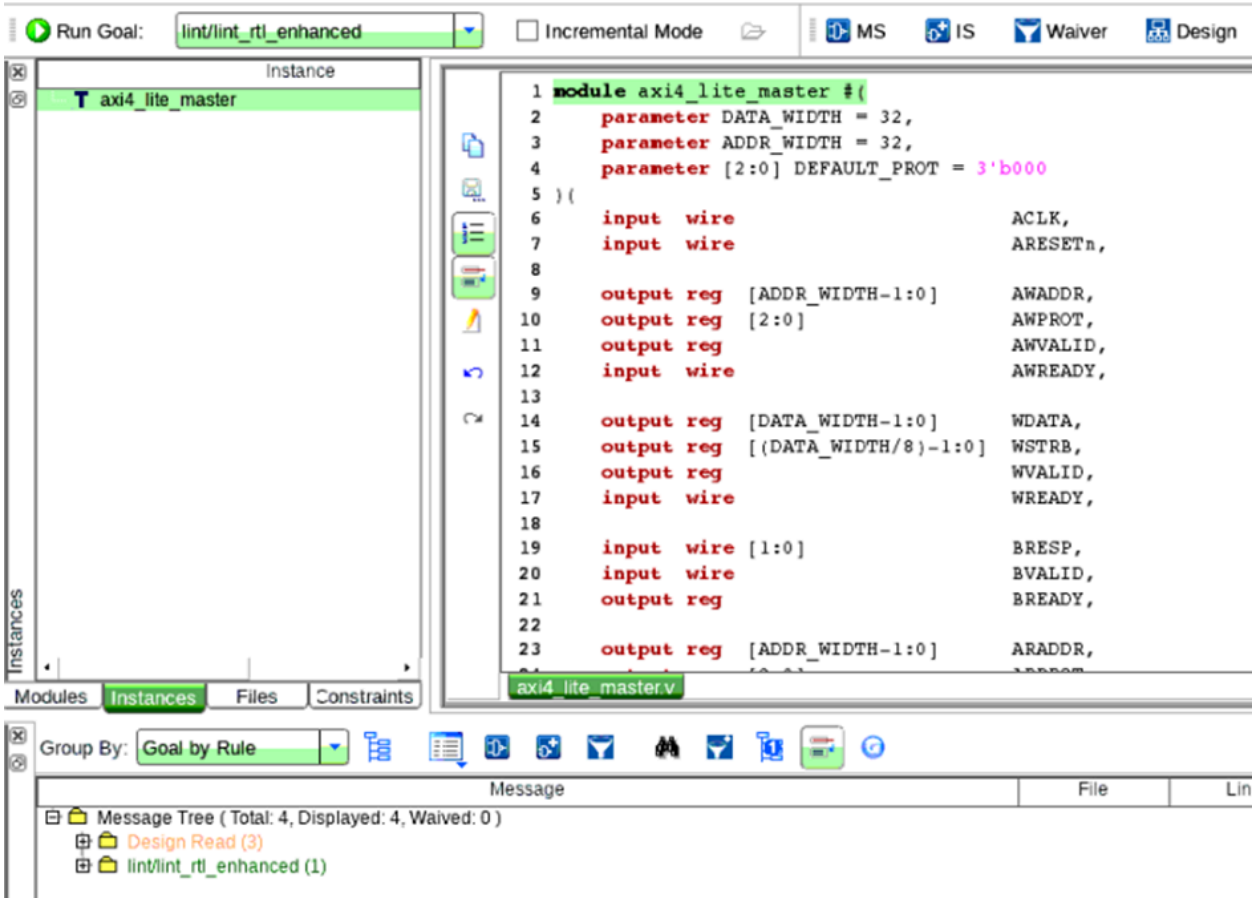
endmodule
```

## 11.2 Lint Reports

### Slave IP



## Master IP


Run Goal:
lint/lint_rtl_enhanced
Incremental Mode
MS
IS
Waiver
Design
Instance
axi4_lite_master
Instances
Modules
Instances
Files
Constraints
1 module axi4_lite_master #(
2 parameter DATA_WIDTH = 32,
3 parameter ADDR_WIDTH = 32,
4 parameter [2:0] DEFAULT_PROT = 3'b000
5 )(
6 input wire ACLK,
7 input wire ARESETn,
8
9 output reg [ADDR_WIDTH-1:0] AWADDR,
10 output reg [2:0] AWPROT,
11 output reg AWVALID,
12 input wire AWREADY,
13
14 output reg [DATA_WIDTH-1:0] WDATA,
15 output reg [(DATA_WIDTH/8)-1:0] WSTRB,
16 output reg WVALID,
17 input wire WREADY,
18
19 input wire [1:0] BRESP,
20 input wire BVALID,
21 output reg BREADY,
22
23 output reg [ADDR_WIDTH-1:0] ARADDR,
axi4_lite_master.v
Group By:
Goal by Rule
Message
File
Lin
Message Tree ( Total: 4, Displayed: 4, Waived: 0 )
Design Read (3)
lint/lint_rtl_enhanced (1)

## 11.3 Timing Reports

### Slave IP Setup Timing

```
Report : timing
         -path full
         -delay max
         -max_paths 1
Design : axi4_lite_slave
Version: S-2021.06-SP4
Date   : Mon Apr 27 10:51:24 2026
****************************************

Operating Conditions: ss0p75v25c   Library: saed32lvt_ss0p75v25c
Wire Load Model Mode: enclosed

  Startpoint: AWADDR[7] (input port clocked by ACLK)
  Endpoint: reg_file_reg[0][0]
            (rising edge-triggered flip-flop clocked by ACLK)
  Path Group: ACLK
  Path Type: max

  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  axi4_lite_slave    8000                  saed32lvt_ss0p75v25c

  Point                                    Incr       Path
  -----------------------------------------------------------
  clock ACLK (rise edge)                   0.00       0.00
  clock network delay (ideal)              0.00       0.00
  input external delay                     4.00       4.00 f
  AWADDR[7] (in)                           0.00       4.00 f
  U1121/Y (OR4X1_LVT)                      0.19       4.19 f
  U792/Y (NOR4X1_LVT)                      0.19       4.38 r
  U791/Y (NAND4X0_LVT)                     0.08       4.46 f
  U786/Y (AO21X1_LVT)                      0.09       4.54 f
  U785/Y (OR3X1_LVT)                       0.09       4.64 f
  U729/Y (NOR2X0_LVT)                      0.11       4.75 r
  U1043/Y (AND3X1_LVT)                     0.16       4.91 r
  U1056/Y (NAND2X0_LVT)                    0.13       5.04 f
  U894/Y (INVX1_LVT)                       0.13       5.17 r
  U681/Y (AO22X1_LVT)                      0.09       5.26 r
  reg_file_reg[0][0]/D (DFFARX1_LVT)       0.00       5.26 r
  data arrival time                                   5.26

  clock ACLK (rise edge)                  10.00      10.00
  clock network delay (ideal)              0.00      10.00
  clock uncertainty                       -1.00       9.00
  reg_file_reg[0][0]/CLK (DFFARX1_LVT)     0.00       9.00 r
  library setup time                      -0.08       8.92
  data required time                                  8.92
  -----------------------------------------------------------
  data required time                                  8.92
  data arrival time                                  -5.26
  -----------------------------------------------------------
  slack (MET)                                        3.66
```

### Slave IP Hold Timing

```
dc_shell> report_timing -delay_type min

****************************************
Report : timing
        -path full
        -delay min
        -max_paths 1
Design : axi4_lite_slave
Version: S-2021.06-SP4
Date   : Mon Apr 27 10:51:51 2026
****************************************

Operating Conditions: ss0p75v25c   Library: saed32lvt_ss0p75v25c
Wire Load Model Mode: enclosed

  Startpoint: rd_state_reg
              (rising edge-triggered flip-flop clocked by ACLK)
  Endpoint: ARREADY_reg
            (rising edge-triggered flip-flop clocked by ACLK)
  Path Group: ACLK
  Path Type: min

  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  axi4_lite_slave    8000                  saed32lvt_ss0p75v25c

  Point                                    Incr       Path
  -----------------------------------------------------------
  clock ACLK (rise edge)                   0.00       0.00
  clock network delay (ideal)              0.00       0.00
  rd_state_reg/CLK (DFFARX1_LVT)           0.00       0.00 r
  rd_state_reg/QN (DFFARX1_LVT)            0.14       0.14 f
  U3/Y (AO22X1_LVT)                        0.09       0.22 f
  ARREADY_reg/D (DFFARX1_LVT)              0.00       0.22 f
  data arrival time                                   0.22

  clock ACLK (rise edge)                   0.00       0.00
  clock network delay (ideal)              0.00       0.00
  ARREADY_reg/CLK (DFFARX1_LVT)            0.00       0.00 r
  library hold time                       -0.03      -0.03
  data required time                                 -0.03
  -----------------------------------------------------------
  data required time                                 -0.03
  data arrival time                                  -0.22
  -----------------------------------------------------------
  slack (MET)                                         0.25
```

**Master IP Setup Timing**

```
        -path full
        -delay max
        -max_paths 1
Design : axi4_lite_master
Version: S-2021.06-SP4
Date   : Tue Apr 28 13:53:24 2026
****************************************

Operating Conditions: ss0p75v25c   Library: saed32lvt_ss0p75v25c
Wire Load Model Mode: enclosed

  Startpoint: AWREADY (input port clocked by ACLK)
  Endpoint: BREADY_reg (rising edge-triggered flip-flop clocked by ACLK)
  Path Group: ACLK
  Path Type: max

  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  axi4_lite_master   8000                  saed32lvt_ss0p75v25c

  Point                                    Incr       Path
  -----------------------------------------------------------
  clock ACLK (rise edge)                   0.00       0.00
  clock network delay (ideal)              0.00       0.00
  input external delay                     4.00       4.00 f
  AWREADY (in)                             0.00       4.00 f
  U204/Y (NAND2X0_LVT)                     0.06       4.06 r
  U242/Y (INVX1_LVT)                       0.03       4.08 f
  U201/Y (AOI21X1_LVT)                     0.14       4.23 r
  U200/Y (OA21X1_LVT)                      0.08       4.31 r
  U199/Y (NAND3X0_LVT)                     0.07       4.38 f
  U264/Y (INVX1_LVT)                       0.05       4.43 r
  U197/Y (AO22X1_LVT)                      0.09       4.52 r
  BREADY_reg/D (DFFARX1_LVT)               0.00       4.52 r
  data arrival time                                   4.52

  clock ACLK (rise edge)                  10.00      10.00
  clock network delay (ideal)              0.00      10.00
  clock uncertainty                       -1.00       9.00
  BREADY_reg/CLK (DFFARX1_LVT)             0.00       9.00 r
  library setup time                      -0.08       8.92
  data required time                                  8.92
  -----------------------------------------------------------
  data required time                                  8.92
  data arrival time                                  -4.52
  -----------------------------------------------------------
  slack (MET)                                         4.40
```

**Master IP Hold Timing**

```
dc_shell> report_timing -delay_type min

****************************************
Report : timing
        -path full
        -delay min
        -max_paths 1
Design : axi4_lite_master
Version: S-2021.06-SP4
Date   : Tue Apr 28 13:55:08 2026
****************************************

Operating Conditions: ss0p75v25c   Library: saed32lvt_ss0p75v25c
Wire Load Model Mode: enclosed

  Startpoint: ARADDR_reg[0]
              (rising edge-triggered flip-flop clocked by ACLK)
  Endpoint: ARADDR_reg[0]
            (rising edge-triggered flip-flop clocked by ACLK)
  Path Group: ACLK
  Path Type: min

  Des/Clust/Port     Wire Load Model       Library
  ------------------------------------------------
  axi4_lite_master   8000                  saed32lvt_ss0p75v25c

  Point                                    Incr       Path
  -----------------------------------------------------------
  clock ACLK (rise edge)                   0.00       0.00
  clock network delay (ideal)              0.00       0.00
  ARADDR_reg[0]/CLK (DFFARX1_LVT)          0.00       0.00 r
  ARADDR_reg[0]/Q (DFFARX1_LVT)            0.19       0.19 f
  U21/Y (AO22X1_LVT)                       0.07       0.26 f
  ARADDR_reg[0]/D (DFFARX1_LVT)            0.00       0.26 f
  data arrival time                                   0.26

  clock ACLK (rise edge)                   0.00       0.00
  clock network delay (ideal)              0.00       0.00
  ARADDR_reg[0]/CLK (DFFARX1_LVT)          0.00       0.00 r
  library hold time                       -0.03      -0.03
  data required time                                 -0.03
  -----------------------------------------------------------
  data required time                                 -0.03
  data arrival time                                  -0.26
  -----------------------------------------------------------
  slack (MET)                                         0.29
```